Labo5: Dynamic parallelism and asynchronous operation

Arthur Valentin, Joeri Sevens

November 29, 2024

Overview

In dit labo onderzoeken we het verschil tussen synchrone en asynchrone uitvoering van een programma om de efficiëntie van beide benaderingen te vergelijken. Het experiment richt zich op verschillende operaties (som, product, minimum en maximum) uitgevoerd op vier onafhankelijke arrays. Deze arrays worden gegenereerd met een for-lus en verwerkt op de GPU.

Bij de synchrone aanpak worden de arrays één voor één gegenereerd en volledig verwerkt voordat de volgende array wordt gegenereerd en verzonden naar de GPU. Bij de asynchrone aanpak worden de arrays gegenereerd terwijl de GPU bezig is met de verwerking van eerder verzonden data, waardoor CPU- en GPU-taken overlappen.

Het doel is om de totale uitvoeringstijd, inclusief CPU-verwerking, te meten en te vergelijken tussen beide methoden, en te bepalen hoe asynchrone uitvoering de efficiëntie van een GPU-accelerated workflow kan verbeteren.

Methodology

Onze implementatie bouwt voort op de code van labo 2, specifiek de reduced max kernell. Vanuit deze kernell zijn we ook begonnen om een reduces min, som en product kernell te genereren. We definiëren en vullen eerst een array op de CPU en migreren deze vervolgens naar de GPU voor verwerking in een kernel-functie. Dit proces wordt vier keer herhaald, één keer voor elke operatie: som, product, minimum en maximum.

We hebben twee CPU-functies geïmplementeerd:

Een synchrone functie, die de kernels één voor één uitvoert, telkens wachtend tot de vorige volledig afgerond is. Een asynchrone functie, die gegevens voorbereidt voor de volgende kernel terwijl de huidige kernel op de GPU wordt uitgevoerd. In de main()-functie roepen we beide CPU-functies aan en meten we de uitvoertijd met de Chrono-bibliotheek. We synchroniseren de GPU expliciet voordat de timer stopt om een nauwkeurige meting te garanderen.

We merkten dat de getallen in de product kernel snel groter werden dan het maximale bereik van een ‘long long int‘. Daarom hebben we ervoor gekozen om de sum kernel tweemaal uit te voeren op verschillende arrays in plaats van de product kernel.

Conclusion

Uit de resultaten blijkt duidelijk dat asynchrone uitvoering sneller is dan synchrone uitvoering (zie figuur 1). Dit komt doordat bij de asynchrone aanpak de CPU en GPU taken overlappen: terwijl de GPU een array verwerkt, kan de CPU al beginnen met het genereren van de volgende array. Hierdoor wordt de wachttijd geminimaliseerd en de totale uitvoeringstijd verkort, in tegenstelling tot de synchrone aanpak, waar elke array moet wachten tot de volledige verwerking van de vorige array is afgerond.

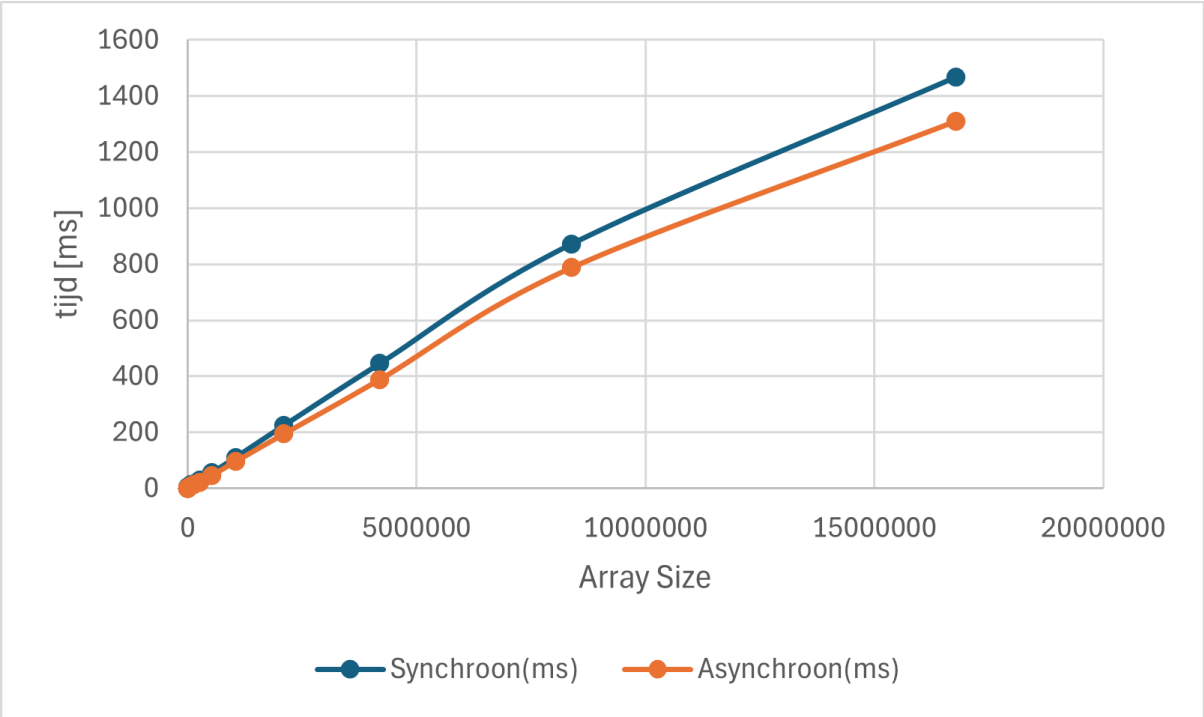


Figure 1: Uitvoeringstijd als functie van de grootte van array

ArraySize	Synchronous(ms)	Asynchronous(ms)
1	0.253984	0.070624
2	0.149568	0.066528
4	0.132096	0.06864
8	0.173056	0.073728
16	0.173984	0.067712
32	0.135136	0.064512
64	0.149536	0.075808
128	0.774176	0.096128
256	0.161792	0.083904
512	0.203744	0.104352
1024	0.24272	0.14528
2048	0.32576	0.228352
4096	0.525312	0.393216
8192	5.3464	0.743456
16384	2.50979	2.93786
32768	4.88054	4.3632
65536	13.1809	7.58992
131072	16.1372	12.672
262144	30.1425	22.6939
524288	55.3462	46.2439
1048576	110.946	95.9171
2097152	224.048	193.922
4194304	445.37	388.028
8388608	872.641	787.616
16777216	1467.97	1309.73

Table 1: synchroon VS assynchroon tijden

Code nested CPU

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <math_constants.h>
#include <chrono>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <fstream>
#include <cmath>

using namespace std;

__global__ void reduceSum(long long int* A, long long int* B, int N) {
    extern __shared__ long long int sdata[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    sdata[tid] = (i < N) ? A[i] : 0;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) B[blockIdx.x] = sdata[0];
}

__global__ void reduceMin(long long int* A, long long int* B, int N) {
    extern __shared__ long long int sdata[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    sdata[tid] = (i < N) ? A[i] : LLONG_MAX;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = min(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }

    if (tid == 0) B[blockIdx.x] = sdata[0];
}

__global__ void reduceMax(long long int* A, long long int* B, int N) {
    extern __shared__ long long int sdata[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int tid = threadIdx.x;

    sdata[tid] = (i < N) ? A[i] : LLONG_MIN;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = max(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }
```

```
63
64      if (tid == 0) B[blockIdx.x] = sdata[0];
65  }
66
67  // Initialize array with random numbers
68  void initializeArray(long long int* arr, int N) {
69      for (int i = 0; i < N; i++) {
70          arr[i] = (rand() % 99) + 1;
71      }
72  }
73
74  // Function to print results
75  void printResult(const char* operation, long long int* d_result) {
76      long long int result;
77      cudaMemcpy(&result, d_result, sizeof(long long int), cudaMemcpyDeviceToHost);
78      printf("%s result: %lld\n", operation, result);
79  }
80
81  int main() {
82      // Initialize CUDA
83      cudaSetDevice(0);
84
85      ofstream csvFile;
86      csvFile.open("comparison_results.csv");
87      csvFile << "ArraySize,Synchronous(ms),Asynchronous(ms)\n";
88
89      for (int exp = 0; exp <= 24; ++exp) { // Loop from 2^0 to 2^14 (1 to 16384
            elements)
90          int N = 1 << exp; // 2^exp
91          cout << "itteratie: " << exp << endl;
92
93          long long int *h_A1, *h_A2, *h_A3, *h_A4;
94          long long int *d_A1, *d_A2, *d_A3, *d_A4;
95          long long int *d_B1, *d_B2, *d_B3, *d_B4;
96          long long int *d_result1, *d_result2, *d_result3, *d_result4;
97
98          h_A1 = (long long int*)malloc(N * sizeof(long long int));
99          h_A2 = (long long int*)malloc(N * sizeof(long long int));
100         h_A3 = (long long int*)malloc(N * sizeof(long long int));
101         h_A4 = (long long int*)malloc(N * sizeof(long long int));
102
103         cudaMalloc(&d_A1, N * sizeof(long long int));
104         cudaMalloc(&d_A2, N * sizeof(long long int));
105         cudaMalloc(&d_A3, N * sizeof(long long int));
106         cudaMalloc(&d_A4, N * sizeof(long long int));
107
108         cudaMalloc(&d_B1, N * sizeof(long long int));
109         cudaMalloc(&d_B2, N * sizeof(long long int));
110         cudaMalloc(&d_B3, N * sizeof(long long int));
111         cudaMalloc(&d_B4, N * sizeof(long long int));
112
113         cudaMalloc(&d_result1, sizeof(long long int));
114         cudaMalloc(&d_result2, sizeof(long long int));
115         cudaMalloc(&d_result3, sizeof(long long int));
116         cudaMalloc(&d_result4, sizeof(long long int));
117
118         int blocksPerGrid = (N + 1024 - 1) / 1024;
119         int sharedMemSize = 1024 * sizeof(long long int);
120
121         cudaEvent_t start, stop;
122         cudaEventCreate(&start);
123         cudaEventCreate(&stop);
124
```

```
        cudaStream_t stream1, stream2, stream3, stream4;
        cudaStreamCreate(&stream1);
        cudaStreamCreate(&stream2);
        cudaStreamCreate(&stream3);
        cudaStreamCreate(&stream4);

        float syncMilliseconds = 0;
        float asyncMilliseconds = 0;

        // Synchronous execution
        cudaEventRecord(start);

        // 1st array and sum
        initializeArray(h_A1, N);
        cudaMemcpy(d_A1, h_A1, N * sizeof(long long int), cudaMemcpyHostToDevice);
        reduceSum<<<blocksPerGrid, 1024, sharedMemSize>>>(d_A1, d_B1, N);
        cudaDeviceSynchronize();
        reduceSum<<<1, 1024, sharedMemSize>>>(d_B1, d_result1, blocksPerGrid);
        cudaDeviceSynchronize();

        // 2nd array and sum
        initializeArray(h_A2, N);
        cudaMemcpy(d_A2, h_A2, N * sizeof(long long int), cudaMemcpyHostToDevice);
        reduceSum<<<blocksPerGrid, 1024, sharedMemSize>>>(d_A2, d_B2, N);
        // Synchronous execution continued...
        cudaDeviceSynchronize();
        reduceSum<<<1, 1024, sharedMemSize>>>(d_B2, d_result2, blocksPerGrid);
        cudaDeviceSynchronize();

        // 3rd array and min
        initializeArray(h_A3, N);
        cudaMemcpy(d_A3, h_A3, N * sizeof(long long int), cudaMemcpyHostToDevice);
        reduceMin<<<blocksPerGrid, 1024, sharedMemSize>>>(d_A3, d_B3, N);
        cudaDeviceSynchronize();
        reduceMin<<<1, 1024, sharedMemSize>>>(d_B3, d_result3, blocksPerGrid);
        cudaDeviceSynchronize();

        // 4th array and max
        initializeArray(h_A4, N);
        cudaMemcpy(d_A4, h_A4, N * sizeof(long long int), cudaMemcpyHostToDevice);
        reduceMax<<<blocksPerGrid, 1024, sharedMemSize>>>(d_A4, d_B4, N);
        cudaDeviceSynchronize();
        reduceMax<<<1, 1024, sharedMemSize>>>(d_B4, d_result4, blocksPerGrid);
        cudaDeviceSynchronize();

        // Record the overall stop time for synchronous execution
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&syncMilliseconds, start, stop);

        // Asynchronous execution
        cudaEventRecord(start);

        // 1st array and sum
        initializeArray(h_A1, N);
        cudaMemcpyAsync(d_A1, h_A1, N * sizeof(long long int),
            cudaMemcpyHostToDevice, stream1);
        reduceSum<<<blocksPerGrid, 1024, sharedMemSize, stream1>>>(d_A1, d_B1, N);
        reduceSum<<<1, 1024, sharedMemSize, stream1>>>(d_B1, d_result1,
            blocksPerGrid);

        // 2nd array and sum
        initializeArray(h_A2, N);
```

```
        cudaMemcpyAsync(d_A2, h_A2, N * sizeof(long long int),
            cudaMemcpyHostToDevice, stream2);
        reduceSum<<<blocksPerGrid, 1024, sharedMemSize, stream2>>>(d_A2, d_B2, N);
        reduceSum<<<1, 1024, sharedMemSize, stream2>>>(d_B2, d_result2,
            blocksPerGrid);

        // 3rd array and min
        initializeArray(h_A3, N);
        cudaMemcpyAsync(d_A3, h_A3, N * sizeof(long long int),
            cudaMemcpyHostToDevice, stream3);
        reduceMin<<<blocksPerGrid, 1024, sharedMemSize, stream3>>>(d_A3, d_B3, N);
        reduceMin<<<1, 1024, sharedMemSize, stream3>>>(d_B3, d_result3,
            blocksPerGrid);

        // 4th array and max
        initializeArray(h_A4, N);
        cudaMemcpyAsync(d_A4, h_A4, N * sizeof(long long int),
            cudaMemcpyHostToDevice, stream4);
        reduceMax<<<blocksPerGrid, 1024, sharedMemSize, stream4>>>(d_A4, d_B4, N);
        reduceMax<<<1, 1024, sharedMemSize, stream4>>>(d_B4, d_result4,
            blocksPerGrid);

        // Synchronize streams
        cudaStreamSynchronize(stream1);
        cudaStreamSynchronize(stream2);
        cudaStreamSynchronize(stream3);
        cudaStreamSynchronize(stream4);

        // Record the overall stop time for asynchronous execution
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&asyncMilliseconds, start, stop);

        // Write the results to the CSV file
        csvFile << N << "," << syncMilliseconds << "," << asyncMilliseconds << "\n"
            ;

        // Clean up memory for this loop iteration
        cudaFree(d_A1);
        cudaFree(d_A2);
        cudaFree(d_A3);
        cudaFree(d_A4);
        cudaFree(d_B1);
        cudaFree(d_B2);
        cudaFree(d_B3);
        cudaFree(d_B4);
        cudaFree(d_result1);
        cudaFree(d_result2);
        cudaFree(d_result3);
        cudaFree(d_result4);

        free(h_A1);
        free(h_A2);
        free(h_A3);
        free(h_A4);

        cudaEventDestroy(start);
        cudaEventDestroy(stop);
        cudaStreamDestroy(stream1);
        cudaStreamDestroy(stream2);
        cudaStreamDestroy(stream3);
        cudaStreamDestroy(stream4);
    }
```

```
242
243     csvFile.close();
244     return 0;
245 }
```